1

```
module queue();

    int j = 1;
    int queue[$] = {0,2,5};

    initial begin
        $display ("size=%0d queue=%p", queue.size(), queue);
        queue.insert(1,j);
        $display ("size=%0d queue=%p", queue.size(), queue);
        queue.delete(1);
        $display ("size=%0d queue=%p", queue.size(), queue);
        queue.push_front(7);
        $display ("size=%0d queue=%p", queue.size(), queue);
        queue.push_back(9);
        $display ("size=%0d queue=%p", queue.size(), queue);
        j = queue.pop_back();
        $display ("size=%0d queue=%p j = %0d", queue.size(), queue, j);
        j = queue.pop_front();
        $display ("size=%0d queue=%p j = %0d", queue.size(), queue, j);
        queue.sort();
        $display ("queue after sorting: %p ",queue);
        queue.reverse();
        $display ("queue after reversed: %p ",queue);
        queue.shuffle();
        $display ("queue after shuffled: %p ",queue);
    end

endmodule
```

```
# size=3 queue='{0, 2, 5}
# size=4 queue='{0, 1, 2, 5}
# size=3 queue='{0, 2, 5}
# size=4 queue='{7, 0, 2, 5}
# size=5 queue='{7, 0, 2, 5, 9}
# size=4 queue='{7, 0, 2, 5} j = 9
# size=3 queue='{0, 2, 5} j = 7
# queue after sorting: '{0, 2, 5}
# queue after reversed: '{5, 2, 0}
# queue after shuffled: '{5, 2, 0}
```

2

```
package adder_pkg;

    typedef enum bit signed [3:0] {MAXNEG = 4'sb1000, ZERO = 4'sb000, MAXPOS = 4'sb0111} val_t;

    class adder_transaction;

      bit clk;
      rand bit reset;
      rand bit signed [3:0] A;
      rand bit signed [3:0] B;
      rand val_t AB_val;

        constraint reset_c{
            reset dist {1 := 2, 0 := 98};
        }

        constraint A_B_c{
            A dist{AB_val :/ 75, [-7:6] :/ 25};
            B dist{AB_val :/ 75, [-7:6] :/ 25};
        }

        covergroup A_cvg @(posedge clk);

            Covgrp_A: coverpoint A{
                bins A_data_0 = {ZERO};
                bins A_data_min = {MAXNEG};
                bins A_data_max = {MAXPOS};
                bins A_data_default = default;
            }

            Covpt_A_trans: coverpoint A{
                bins A_data_0max = (ZERO => MAXPOS);
                bins A_data_maxmin = (MAXPOS => MAXNEG);
                bins A_data_minmax = (MAXNEG => MAXPOS);
            }

        endgroup
```

```
        covergroup B_cvg @(posedge clk);

            Covgrp_B: coverpoint B{
                bins B_data_0 = {ZERO};
                bins B_data_min = {MAXNEG};
                bins B_data_max = {MAXPOS};
                bins B_data_default = default;
            }

            Covpt_B_trans: coverpoint B{
                bins B_data_0max = (ZERO => MAXPOS);
                bins B_data_maxmin = (MAXPOS => MAXNEG);
                bins B_data_minmax = (MAXNEG => MAXPOS);
            }

        endgroup

        function new();
                A_cvg = new();
                B_cvg = new();
        endfunction

        task sample_inputs();
            if(!reset) begin
                A_cvg.sample();
                B_cvg.sample();
            end
        endtask

    endclass

endpackage
```

```
import adder_pkg::*;

module adder_tb();

    adder_transaction addr_obj;

    // declare local input and outptut
    logic clk, reset;
    logic signed [3:0] A, B;
    wire signed [4:0] C;

    bit signed [4:0] C_exp;

    // Declare error_count and correct_count as integer.
    int error_count, correct_count;

    // Instantiate the design under test (adder).
    adder uut(
        clk,
        reset,
        A,
        B,
        C
    );

    // Write a clock generator block that toggles clk_tb every 5 time units.
    initial begin
        clk =1'b0;
        forever begin
            #5 clk = ~clk;
            addr_obj.clk = clk;
        end
    end
```

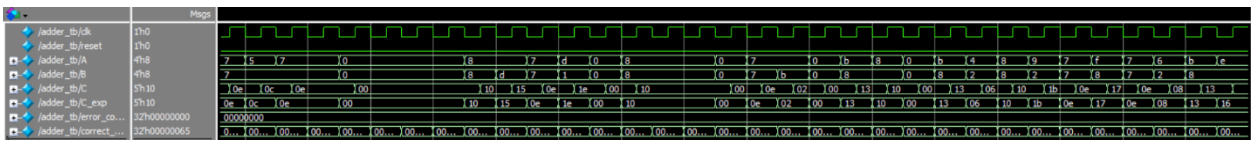
```
// General stimuli
initial begin

  addr_obj = new();
  reset = 1'b0;
  A = 4'b0;
  B = 4'b0;

  #2;
  reset = 1'b1;

  assert_reset;

  repeat(100) begin
    assert(addr_obj.randomize());
    reset = addr_obj.reset;
    A = addr_obj.A;
    B = addr_obj.B;
    @(posedge clk);
    addr_obj.sample_inputs();
    check_result(addr_obj);
  end
  $stop;
end

task assert_reset;
    reset = 1;
    check_reset;
    reset = 0;
endtask
```

```
    task check_reset;
        @(negedge clk);
        if (C == 0) begin
            correct_count += 1;
            $display("reset asserted");
        end else begin
            error_count += 1;
            $display("reset failed");
        end
    endtask

    task check_result(adder_transaction addr_obj1);
        golden_model(addr_obj1);
        @(negedge clk);
        if(!addr_obj1.reset) begin
            if (C == C_exp) begin
                correct_count += 1;
                $display("correct_count = %0d at %t: C = %0d and C_exp = %0d",correct_count, $time, C, C_exp);
            end else begin
                error_count += 1;
                $display("error_count = %0d at %t: C = %0d and C_exp = %0d",error_count, $time, C, C_exp);
            end
        end
    endtask

    task golden_model(adder_transaction addr_obj2);
        if(addr_obj2.reset) begin
            C_exp = 0;
        end else begin
            C_exp = addr_obj2.A + addr_obj2.B;
        end
    endtask


endmodule
```

```
================================================================================
=== Instance: /adder_tb/uut
=== Design Unit: work.adder
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         2         2         0   100.00%

================================Branch Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       3         3         0   100.00%

================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================
```



/adder_pkg/adder_transaction	100.00%				
TYPE A_cvg	100.00%	100	100.00%	auto(0)	
CVP A_cvg::Covgrp_A	100.00%	100	100.00%		
CVP A_cvg::Covpt_A_trans	100.00%	100	100.00%		
INST \/adder_pkg::adder_transaction::A_cvg	100.00%	100	100.00%		0
CVP Covgrp_A	100.00%	100	100.00%		
bin A_data_0	29	1	100.00%		
bin A_data_min	25	1	100.00%		
bin A_data_max	22	1	100.00%		
default bin A_data_default	25	-	-		
CVP Covpt_A_trans	100.00%	100	100.00%		
bin A_data_0max	6	1	100.00%		
bin A_data_maxmin	2	1	100.00%		
bin A_data_minmax	5	1	100.00%		
TYPE B_cvg	100.00%	100	100.00%	auto(0)	
CVP B_cvg::Covgrp_B	100.00%	100	100.00%		
CVP B_cvg::Covpt_B_trans	100.00%	100	100.00%		
INST \/adder_pkg::adder_transaction::B_cvg	100.00%	100	100.00%		0
CVP Covgrp_B	100.00%	100	100.00%		
bin B_data_0	28	1	100.00%		
bin B_data_min	32	1	100.00%		
bin B_data_max	21	1	100.00%		
default bin B_data_default	20	-	-		
CVP Covpt_B_trans	100.00%	100	100.00%		
bin B_data_0max	5	1	100.00%		
bin B_data_maxmin	5	1	100.00%		
bin B_data_minmax	5	1	100.00%		

3

```
package FSM_pkg;

    typedef enum bit [1:0] {IDLE, ZERO, ONE, STORE} state_e;

    class fsm_transaction;
        rand bit rst, x;
        bit y_exp;
        bit [9:0] users_count_exp;

        bit clk;

        constraint rst_cst{
            rst dist {0:= 97, 1:=3};
        }

        constraint x_cst{
            x dist {0:= 67, 1:=33};
        }

        covergroup fsm_cfg @(posedge clk);
            x_cvp: coverpoint x{
                bins x_trans = (0 => 1 => 0);
            }
        endgroup

        function new();
            fsm_cfg = new();
        endfunction

    endclass

endpackage
```

```
import FSM_pkg::*;

module FSM_010_tb();

    //1- Delcare local wir and reg
    logic clk, rst, x;
    logic y;
    logic [9:0] users_count;

    fsm_transaction fsm_obj = new();

    integer error_count, correct_count;

    //2- Instantiate the module under test
    FSM_010 uut(
        clk,
        rst,
        x,
        y,
        users_count);

    //3- Generate clock
    initial begin
        clk = 1'b0;
        forever begin
            #5 clk = ~clk;
            fsm_obj.clk = clk;
        end
    end
```

```
//4- Stimulus generation
initial begin
    error_count = 0;
    correct_count = 0;

    // Assert reset
    assert_reset();

    // Generate random transactions
    repeat (1000) begin
        assert(fsm_obj.randomize());
        rst = fsm_obj.rst;
        x = fsm_obj.x;
        check_result(fsm_obj);
    end

    // Generate random transactions
    repeat (10000) begin
        assert(fsm_obj.randomize());
        fsm_obj.rst = 0; // Ensure reset is deasserted for this part of the test
        rst = 0;
        x = fsm_obj.x;
        check_result(fsm_obj);
    end

    $display("Total tests: %d, Passed: %d, Failed: %d", error_count + correct_count, correct_count, error_count);
    $stop;
end
```

```
task golden_model(input fsm_transaction fsm_obj1);
    state_e cs, ns;
    case (cs)
        IDLE:
            if (fsm_obj.x)
                ns = IDLE;
            else
                ns = ZERO;
        ZERO:
            if (fsm_obj.x)
                ns = ONE;
            else
                ns = ZERO;
        ONE:
            if (fsm_obj.x)
                ns = IDLE;
            else
                ns = STORE;
        STORE:
            if (fsm_obj.x)
                ns = IDLE;
            else
                ns = ZERO;
        default:    ns = IDLE;
    endcase
```

```
        @(posedge clk);
            if(fsm_obj.rst) begin
                cs = IDLE;
                fsm_obj.users_count_exp = 0;
            end
            else begin
                cs = ns;
                fsm_obj.users_count_exp =  (cs == STORE) ? fsm_obj.users_count_exp + 1 : fsm_obj.users_count_exp;
            end
        fsm_obj.y_exp = (cs == ONE) ? 1'b1 : 1'b0;
    endtask

    task assert_reset;
        rst = 1'b1;
        check_reset();
        rst = 1'b0;
    endtask

    task check_result(input fsm_transaction fsm_obj2);
        golden_model(fsm_obj2);
        if (!fsm_obj2.rst) begin
            @(negedge clk);
            if ((fsm_obj2.y_exp !== y) || (fsm_obj2.users_count_exp !== users_count)) begin
                error_count += 1;
                $display("Test failed for y.Expected: %b, Y: %b", fsm_obj2.y_exp, y);
            end else begin
                correct_count += 1;
                $display("Test passed for y.Expected: %b, Y: %b", fsm_obj2.y_exp, y);
            end
        end else begin
            check_reset();
        end
    endtask;
```

```
    task check_reset;
        @(negedge clk);
           if ((y !== 0) || (users_count !== 0)) begin
               error_count += 1;
               $display("Reset test failed. Expected y: 0, y: %b, Expected users_count: 0, users_count: %d", y, users_count);
           end else begin
               correct_count += 1;
               $display("Reset test passed. Expected y: 0, y: %b, Expected users_count: 0, users_count: %d", y, users_count);
           end
    endtask


endmodule
```


```
    task check_reset;
        @(negedge clk);
           if ((y !== 0) || (users_count !== 0)) begin
               error_count += 1;
               $display("Reset test failed. Expected y: 0, y: %b, Expected users_count: 0, users_count: %d", y, users_count);
           end else begin
               correct_count += 1;
               $display("Reset test passed. Expected y: 0, y: %b, Expected users_count: 0, users_count: %d", y, users_count);
           end
    endtask


endmodule
```

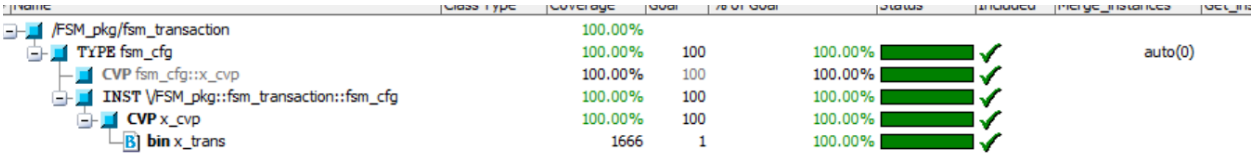
```
================================================================================
=== Instance: /FSM_010_tb/uut
=== Design Unit: work.FSM_010
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        20        20         0   100.00%

================================Branch Details================================
```

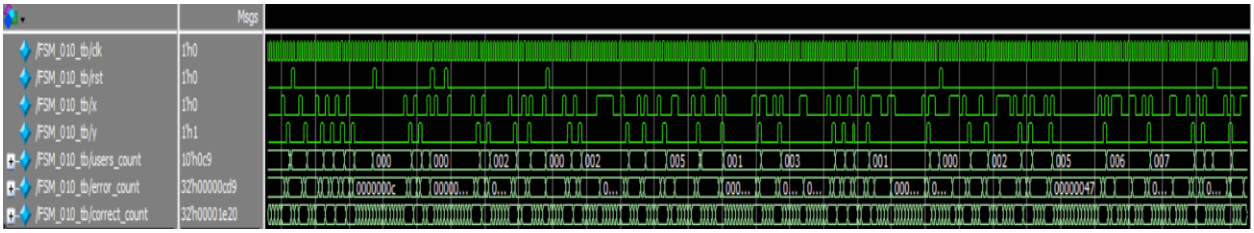
```
  Condition Coverage:
      Enabled Coverage              Bins   Covered    Misses  Coverage
      ----------------              ----      ----    ------  --------
      Conditions                       2         2         0   100.00%

  ================================Condition Details================================

  Condition Coverage for instance /FSM_010_tb/uut --
```

```
FSM Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    FSM States                       4         4         0   100.00%
    FSM Transitions                  7         7         0   100.00%

=================================FSM Details=================================

FSM Coverage for instance /FSM_010_tb/uut --
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      16        16         0   100.00%

================================Statement Details================================

Statement Coverage for instance /FSM_010_tb/uut --
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         36        36         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /FSM_010_tb/uut --
```





4

```
module ass_arr();

    // 24-bit word width, 20-bit address space (2^20 words)
    bit [23:0] mem [bit[19:0]];

    bit [19:0] addr;
    initial begin
        mem[20'h00000] = 24'hA50400; // location 0 to add the reset values
        mem[20'h00400] = 24'h123456; // Instruction 1 located at location 0x400
        mem[20'h00401] = 24'h789ABC; // Instruction 2 located at location 0x401
        mem[20'hFFFFF] = 24'h0F1E2D; // (ISR) is at the maximum possible address.

        foreach(mem[addr]) begin
            $display("Address = 0x%05h Data = 0x%06h",addr, mem[addr]);
        end

    end

endmodule
```

```
# Address = 0x00000 Data = 0xa50400
# Address = 0x00400 Data = 0x123456
# Address = 0x00401 Data = 0x789abc
# Address = 0xfffff Data = 0x0f1e2d
# exit
# End time: 18:23:48 on Apr 13,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
End time: 18:23:48 on Apr 13,2026, Elapsed time: 0:00:01
```

5

```
module struct_test();

    typedef bit [6:0] defined_7bit ;

    typedef struct{
        defined_7bit header;
        defined_7bit cmd;
        defined_7bit data;
        defined_7bit crc;
    } my_packet;

    my_packet packet;
    initial begin
        #2;
        packet.header =7'h5A;
        packet.cmd = 8'h00;
        packet.data = 8'h00;
        packet.crc = 8'h00;

        $display("my_packet = %p",packet);
        $display("header = 0x%0h cmd = 0x%0h data = 0x%0h crc = 0x%0h",
                packet.header,
                packet.cmd,
                packet.data,
                packet.crc);
    end

endmodule
```

```
#
# run -all
# my_packet = '{header:90, cmd:0, data:0, crc:0}
# header = 0x5a cmd = 0x0 data = 0x0 crc = 0x0
# exit
# End time: 18:43:09 on Apr 13,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
End time: 18:43:09 on Apr 13,2026, Elapsed time: 0:00:01
```